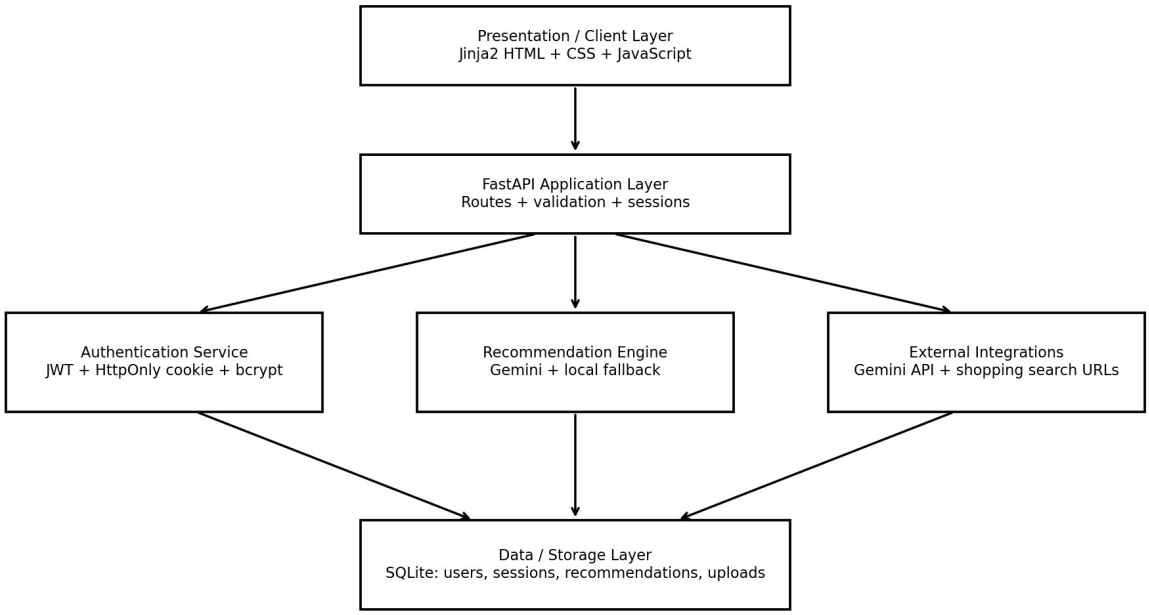
PocketSmart AI – Your Smart Budget & Recommendation Assistant

AI / ML and Generative AI Track – Project Submission Documentation

Prepared in the same phase-wise structure as the supplied project template

Date: 30 September 2026

Team ID: To be filled



1. Brainstorming & Ideation

Brainstorming & Idea Prioritization Template

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 3 Marks

Step 1: Brainstorm and Idea Listing

S.No	Team Member	Idea / Suggestion	Category	Group No.
1	Team – To be filled	AI budget planning assistant for Indian users	Generative AI / Finance	1
2	Team – To be filled	Home interior budget planner	Budget Planning	1
3	Team – To be filled	Party planning and venue recommendation	Planning / Recommendations	1
4	Team – To be filled	Jewelry recommendation from budget and outfit image	Multimodal AI	1
5	Team – To be filled	Recommendation history and shopping links	Personalization / Utility	1
6	Team – To be filled	AI assistant with local fallback when API is unavailable	Reliability	1

Step 2: Idea Prioritization

Group	Final Idea	Feasibility	Importance	Priority	Selected
1	PocketSmart AI – smart budget and recommendation assistant	High	High	1	Yes
2	Generic shopping chatbot	Medium	Medium	2	No
3	Standalone expense tracker	High	Medium	3	No

Selected idea rationale: The selected solution combines budget planning, practical recommendations, generative AI, image-assisted jewelry analysis, and actionable shopping search links in one application.

Define Problem Statements

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 3 Marks

Problem Statement	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	A budget-conscious Indian user	Plan a purchase/event within a fixed budget	Prices and choices are difficult to compare	There are many categories, vendors and options	Confused and time-consuming
PS-2	A homeowner	Plan basic interior purchases	It is difficult to divide a fixed amount across items	Furniture, lights and fans have different costs	Uncertain about priorities
PS-3	A party organizer	Plan venue, catering and decoration	Many cost components must be considered together	Manual planning is fragmented	Overwhelmed
PS-4	A jewelry shopper	Choose jewelry that matches an occasion/outfit	Visual matching and budget planning are separate tasks	Traditional shopping requires browsing many options	Unsure about suitable choices

Empathy Map

Empathy Map

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 4 Marks

SAYS	THINKS	PERSONA / USER TYPE	DOES	FEELS
“I have a fixed budget.” “I want useful recommendations.” “I do not want to overspend.”	“Which items are essential?” “Can AI make a practical plan?” “Can I find products quickly?”	Budget-conscious Indian user / planner	Enters budget and requirements; compares recommendations; opens shopping links; saves/reviews history	Before: confused, uncertain After: informed, organized, more confident

2. Requirement Analysis

Customer Journey Map

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 2 Marks

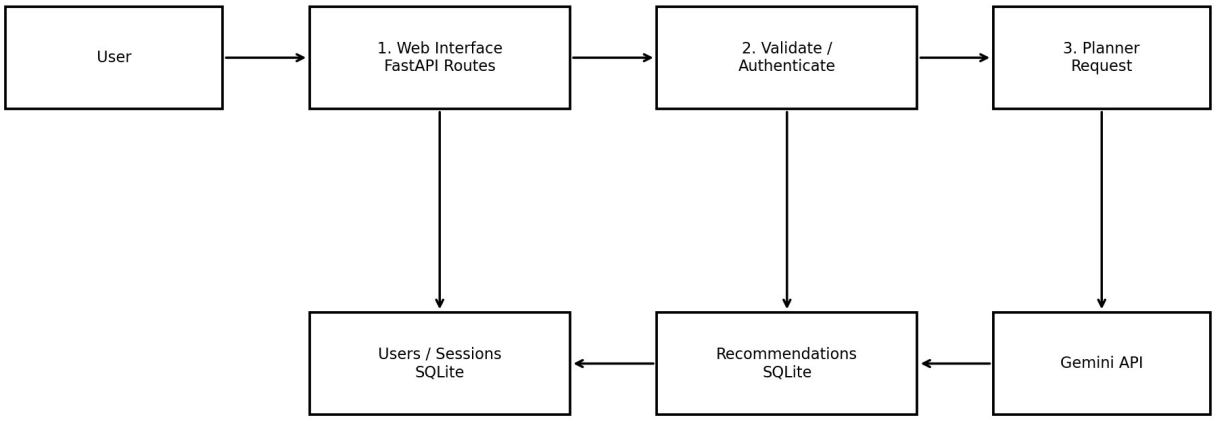
Journey Phase	Stage 1 – Discover	Stage 2 – Plan	Stage 3 – Act / Review
Actions	Open PocketSmart AI; choose planner	Enter budget and requirements; submit	Review plan; open links; revisit history
Touchpoint	Home page / Dashboard	Home, Party or Jewelry planner	Result page / History / shopping links
Customer Thought	“Can this help me?”	“Will this fit my budget?”	“Which recommendation should I explore?”
Customer Feeling	Curious	Concerned but hopeful	More confident and organized
Process Ownership	UI / authentication	FastAPI + recommender + Gemini	Result rendering + history
Opportunities	Clear planner descriptions	Budget validation and AI fallback	Better personalization and saved plans

Data Flow Diagram

Data Flow Diagram

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 2 Marks

PocketSmart AI – DFD Level 0/1 View



DFD interpretation: The user submits authenticated planner data through the web interface. FastAPI validates the request, sends the prompt and optional image to Gemini, receives structured recommendations, normalizes the budget, stores recommendation history, and returns the result to the user. When Gemini is temporarily unavailable, local fallback logic supplies a usable plan.

Solution Requirements

Solution Requirements

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 4 Marks

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	Users can register, log in and log out using secure password hashing and JWT-based sessions stored in an HttpOnly cookie.	High
2	Authorization	Protected dashboard, planner and history pages are available only to authenticated users.	High
3	External interfaces	System integrates with Gemini API and generates search URLs for Indian shopping/service platforms.	High
4	Transactions / processing	Planner inputs are validated, sent to the recommendation engine, normalized to budget, and saved as recommendation history.	High
5	Reporting	Dashboard and history pages display generated plans, spending, remaining budget and recommendation details.	Medium
6	Business rules	Recommendations must stay within the supplied budget; invalid AI output is sanitized; transient AI failures trigger retries/fallback.	High
7	Compliance / privacy	API keys are stored in environment variables; uploaded images are handled by the application and should	High

		be deleted according to the deployment retention policy.	
8	Other	Jewelry planner supports optional outfit-image analysis and multimodal recommendation generation.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Normal pages should load quickly; planner APIs should return within a reasonable interactive time considering external AI latency.	Target page load < 2–3 sec locally; AI response depends on Gemini availability
2	Scalability	Application should allow migration from SQLite to PostgreSQL and deployment behind a production ASGI server.	Support database/server upgrade without redesigning UI
3	Security & Data Privacy	Passwords hashed with bcrypt; JWT stored in HttpOnly cookie; API key kept in .env; input and upload validation applied.	No plaintext passwords/API keys in source code
4	Reliability & Availability	Transient Gemini failures should be retried and a local fallback should prevent planner failure.	HTTP 200 with usable fallback where possible
5	Usability & Accessibility	Responsive HTML interface with clear planner forms, validation and readable result sections.	Usable on desktop/mobile widths
6	Other – Maintainability	Separate modules for config, database, authentication, schemas, recommendation logic and frontend assets.	Modular source structure

Technology Stack

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 2 Marks

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript, Jinja2 templates	Responsive web interface and dynamic result rendering
Backend	Python 3.10, FastAPI, Uvicorn	REST APIs, routing, authentication flow and application server
AI / GenAI	Google Gemini API, google-genai SDK	Budget planning, recommendations and multimodal jewelry analysis
Database	SQLite	Users, sessions, recommendations and upload metadata
Security	python-jose, passlib/bcrypt	JWT tokens and password hashing
Validation	Pydantic / FastAPI validation	Structured and safe API input
Image processing	Pillow / Python file handling	Upload validation and image handling
Development	VS Code, PowerShell, virtual environment	Development and local execution
External discovery	Search URLs for Amazon, Flipkart, Myntra, etc.	Actionable product/service discovery

3. Project Design Phase

Problem-Solution Fit Canvas

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 5 Marks

Canvas Area	PocketSmart AI Content
1. Customer Segment(s) [CS]	Budget-conscious Indian individuals, homeowners planning basic interiors, party/event organizers, and jewelry shoppers.
2. Problems / Pains + Frequency [PR]	Budget uncertainty, excessive browsing, difficulty prioritizing purchases, and lack of coordinated recommendations; occurs whenever a user plans a purchase/event.
3. Triggers to Act [TR]	Fixed budget, upcoming event, home furnishing need, jewelry purchase, or desire for faster decision support.
4. Emotions Before / After [EM]	Before: confusion, uncertainty, time pressure. After: organized options, clearer budget allocation, faster exploration.
5. Available Solutions – Pros & Cons [AS]	Marketplaces provide many products but not personalized budget plans; generic chatbots provide advice but may not structure shopping budgets; manual spreadsheets require user effort.
6. Customer Limitations [CL]	Limited budget, time, product knowledge, and difficulty comparing multiple categories.
7. Behavior + Intensity [BE]	Users search, compare, ask others and browse marketplaces; intensity is high around purchases and events.
8. Channels of Behavior [CH]	Online: websites, marketplaces, search engines. Offline: stores, event vendors, jewelers.
9. Problem Root / Cause [RC]	Information is distributed across categories/vendors and there is no single budget-aware recommendation workflow.
10. Your Solution [SL]	PocketSmart AI combines structured planner forms, Gemini-generated recommendations, budget normalization, shopping links, history, and optional outfit-image analysis.

Proposed Solution

Project Initialization and Planning Phase

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 5 Marks

Project Proposal (Proposed Solution) report

PocketSmart AI is a web-based AI budget and recommendation assistant designed to help Indian users plan home interiors, parties and jewelry purchases within a specified budget. It combines a FastAPI backend, SQLite storage, a responsive web interface, and Google Gemini for generative recommendations. The application also provides shopping/search links and a recommendation history.

Project Overview

Item	PocketSmart AI
Objective	Provide practical, India-specific budget plans and recommendations while reducing manual browsing and helping users stay within a defined budget.
Scope	Authentication, dashboard, Home Interior Planner, Party Planner, Jewelry Planner, optional outfit image analysis, recommendation history, shopping links and AI/local fallback.
Problem Statement	Users often need to coordinate budget, quantities, categories and shopping options manually. Existing marketplaces focus on product discovery rather than complete budget-aware planning.
Impact	The system organizes planning into a single workflow and gives users a structured starting point for decisions.

Proposed Solution

Item	Description
Approach	Collect structured requirements, generate recommendations with Gemini, normalize results to the supplied budget, and provide actionable search links.
Key Features	Secure authentication; three planners; Gemini AI; image-assisted jewelry recommendations; recommendation history; budget tracking; local fallback for transient AI failures.

Resource Requirements

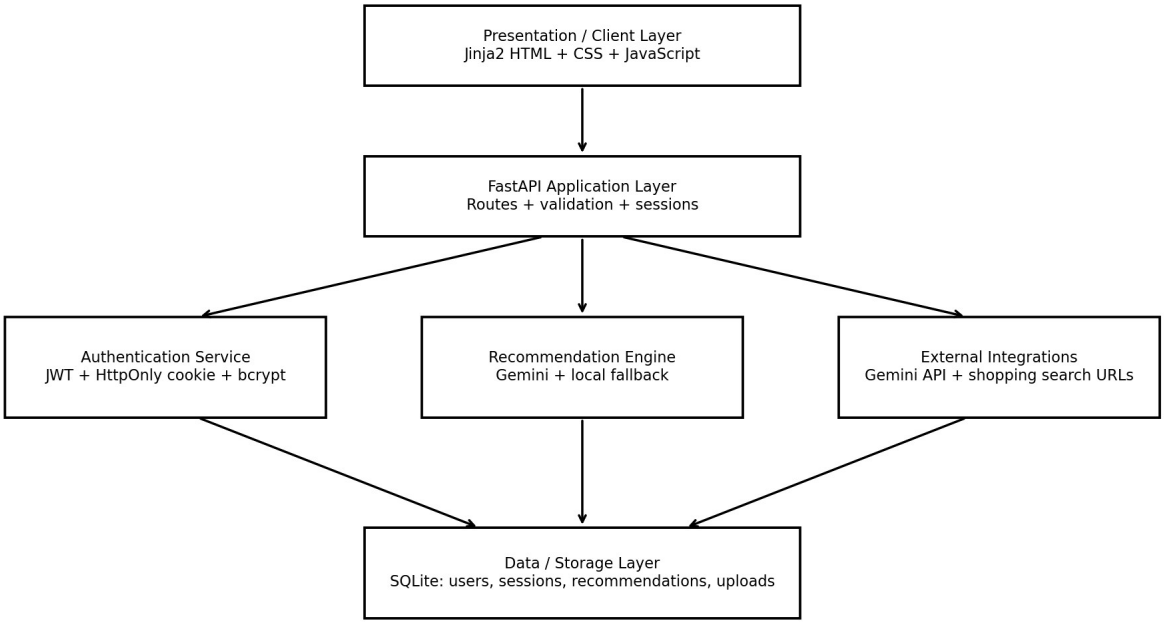
Resource Type	Description	Specification / Allocation
Hardware – Computing	CPU for local FastAPI and AI API calls	Typical modern laptop; dedicated GPU not required for cloud Gemini inference
Memory	Runtime and browser	8 GB RAM or above recommended

Storage	Source, SQLite DB, uploads, logs	At least 2–5 GB free development space
Software – Frameworks	Python web framework	FastAPI + Uvicorn
Libraries	AI, validation, security, templates, image support	google-genai, Pydantic, python-jose, passlib/bcrypt, Jinja2, Pillow
Development Environment	IDE / terminal	VS Code + PowerShell
Data	Application data	User inputs, recommendation JSON, sessions and optional uploaded images

Solution Architecture

Solution Architecture

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 5 Marks



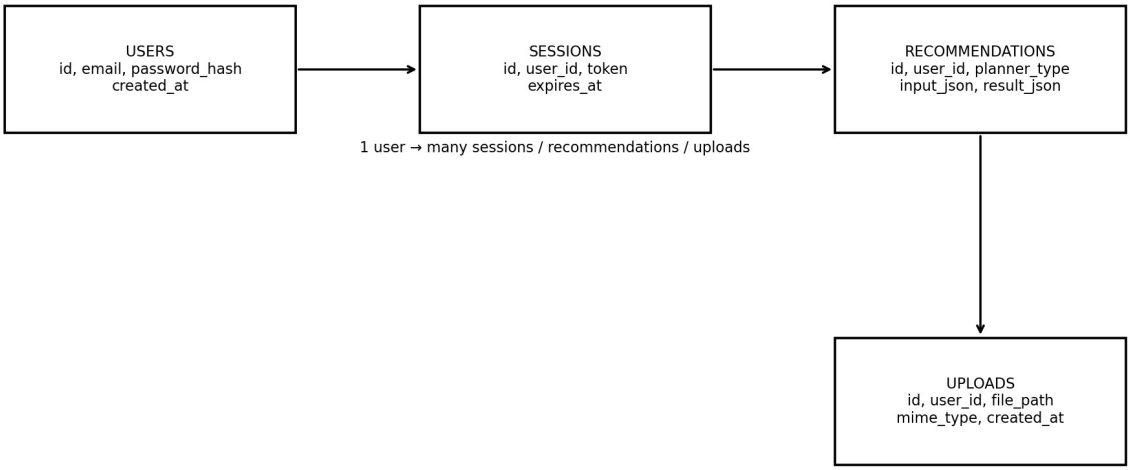
Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Provides landing page, authentication forms, dashboard, planners, results and history.	Jinja2, HTML, CSS, JavaScript
API / Application Layer	Receives HTTP requests, validates inputs and coordinates services.	FastAPI, Uvicorn, Pydantic
Authentication Service	Registers users, hashes passwords, issues JWT sessions and protects routes.	passlib/bcrypt, python-jose, SQLite
Core Logic Service	Builds prompts, calls Gemini, extracts JSON, normalizes budgets and applies local fallback.	Python, google-genai
External API Integrations	Uses Gemini for AI generation and generates external shopping/search URLs.	Google Gemini API, web search URLs

Data / Storage Layer	Stores users, sessions, recommendation history and upload metadata.	SQLite

Database note: Because the solution uses a database, an ER diagram is included as a supplementary design artifact.

PocketSmart AI – ER Diagram (Core Tables)



4. Project Planning Phase

Initial Project Planning Template

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 5 Marks

Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members	Sprint Start	Sprint End (Planned)
Sprint-1	Authentication	USN-1	As a user, I can register with email and password.	2	High	To be filled	To be filled	To be filled
Sprint-1	Authentication	USN-2	As a user, I can log in and log out securely.	2	High	To be filled	To be filled	To be filled
Sprint-1	Dashboard	USN-3	As a user, I can access a dashboard after authentication.	1	High	To be filled	To be filled	To be filled
Sprint-2	Home Planner	USN-4	As a user, I can generate a home interior budget plan.	3	High	To be filled	To be filled	To be filled
Sprint-2	Party Planner	USN-5	As a user, I can generate a party budget and venue suggestions.	3	High	To be filled	To be filled	To be filled
Sprint-3	Jewelry	USN-6	As a user,	3	High	To be	To be	To be

	Planner		I can generate jewelry recommendations from budget and preferences.			filled	filled	filled
Sprint-3	Multimodal AI	USN-7	As a user, I can upload an outfit image for jewelry analysis.	3	Medium	To be filled	To be filled	To be filled
Sprint-4	History	USN-8	As a user, I can view previous recommendations.	2	Medium	To be filled	To be filled	To be filled
Sprint-4	Reliability	USN-9	As a user, I still receive a local plan when Gemini is temporarily unavailable.	2	High	To be filled	To be filled	To be filled

5. Project Development Phase

Code-Layout, Readability and Reusability

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 3 Marks

The application follows a modular structure so that configuration, database access, authentication, schemas, recommendation logic, templates and static assets remain separated.

Module / File	Responsibility	Readability / Reusability Practice
app/main.py	FastAPI routes, page rendering and planner APIs	Routes grouped by responsibility; thin orchestration layer
app/config.py	Environment variables and application configuration	Centralized settings
app/db.py	SQLite initialization and database helpers	Reusable connection/query helpers
app/auth.py	Password hashing, JWT sessions, current-user checks	Authentication isolated from business logic
app/schemas.py	Pydantic request models	Reusable typed validation
app/recommender.py	Gemini calls, prompts, JSON extraction, budget normalization and fallbacks	Shared AI and recommendation functions
templates/	UI pages	Base template and reusable page structure
static/css/styles.css	Visual styling	Central stylesheet
static/js/app.js	Form submission and dynamic result rendering	Shared frontend functions

Suggested Project Layout

```
PocketSmartAI/
├── app/
│   ├── main.py
│   ├── auth.py
│   ├── config.py
│   ├── db.py
│   ├── schemas.py
│   ├── recommender.py
│   ├── templates/
│   └── static/
├── tests/
├── .env
├── requirements.txt
├── README.md
└── run scripts
```

Coding & Solution

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 5 Marks

Implementation Summary

Component	Implementation
Authentication	Registration, password hashing, JWT cookie session, protected routes and logout.
Home Interior Planner	Collects lights, fans, furniture, dining tables, room selections and additional requirements; returns budget breakdown.
Party Planner	Collects party type, guests, venue, catering, decoration and entertainment requirements; returns budget and venue search links.
Jewelry Planner	Uses occasion, preferences and optional outfit image; returns outfit analysis and jewelry recommendations.
Gemini Integration	Uses google-genai client and structured JSON responses. Current implementation retries transient errors and moves across configured current models.
Budget Normalization	Sanitizes prices/quantities and scales recommendations down if generated total exceeds the user budget.
Fallback	Local home, party and jewelry recommendation functions keep the application usable during temporary Gemini unavailability.
History	Recommendation request/result JSON is stored for authenticated users and displayed through the history page.

Current Gemini Reliability Behavior

During development, Gemini 3.8 Flash returned temporary HTTP 503 UNAVAILABLE responses. The application retried and then successfully used another current model, demonstrating the intended resilience behavior. The observed Home Planner API completed with HTTP 200 OK after fallback/model retry.

No. of Functional Features Included in the Solution

Functional Features Included in the Solution

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 5 Marks

S.No	Feature	Description	Status
1	User Registration	Create an account with email/password validation.	Implemented
2	User Login / Logout	Secure authentication using hashed passwords and JWT session cookie.	Implemented
3	Dashboard	Central entry point for planners and history.	Implemented
4	Home Interior Planner	Budget allocation for lights, fans, furniture and rooms.	Implemented
5	Party Planner	Budget allocation for venue, catering, decoration and entertainment.	Implemented
6	Jewelry Planner	Budget-based jewelry recommendations by occasion/preferences.	Implemented
7	Outfit Image Analysis	Optional multimodal image input for jewelry matching.	Implemented / depends on Gemini availability
8	Budget Normalization	Keeps recommendation totals within the supplied budget.	Implemented
9	Shopping Links	Generates marketplace/search links for recommended terms.	Implemented
10	Recommendation History	Stores and displays previous generated plans.	Implemented
11	Gemini Retry / Model Fallback	Retries transient AI failures and tries configured fallback models.	Implemented
12	Local AI Fallback	Provides deterministic recommendations if Gemini cannot respond.	Implemented

Functional feature count: 12 identified features. Exact demonstration count should be updated after the final live demo.

6. Project Testing

Performance Testing

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 5 Marks

Performance testing evaluates how the system behaves under expected and peak load conditions. Formal JMeter/Locust benchmarking has not yet been supplied, so measured CPU, memory, throughput and response-time values are intentionally marked as pending rather than invented.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman / browser developer tools for API functional checks; JMeter or Locust recommended for formal load testing
Type of Testing	Functional API testing and transient-failure resilience testing; formal load/stress test pending
Target Module / API	POST /api/home-budget; POST /api/party-budget; POST /api/jewelry-budget; authentication APIs
Test Environment	Local Windows development environment, FastAPI + Uvicorn
Test Date	30 September 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home planner valid budget request	1	Single request	HTTP 200 and budget result
2	Party planner valid request	1	Single request	HTTP 200 and venue suggestions
3	Jewelry planner with image	1	Single request	HTTP 200 and jewelry analysis/recommendations
4	Gemini temporary 503	1	Several retries	Retry/fallback and usable result without server crash

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 sec for non-AI pages	Not formally benchmarked	Pending	AI latency depends on external Gemini service
2	Response Time (Max)	< 5 sec target for normal	Not formally benchmarked	Pending	Retries can increase AI

		APIs			request duration
3	Throughput	Measure with load tool	Not measured	Pending	Run JMeter/Locust before final submission
4	Error Rate	< 1%	Not benchmarked	Pending	Functional test observed HTTP 200 after AI fallback
5	CPU Utilization	< 80%	Not measured	Pending	Run under representative load
6	Memory Utilization	< 80%	Not measured	Pending	Run under representative load

Step 4: Observations & Analysis

The application remained available when Gemini returned HTTP 503 UNAVAILABLE. The implemented retry and model fallback logic allowed the Home Planner request to finish with HTTP 200 OK. Formal performance metrics require a controlled load test and should be inserted after execution.

7. Project Documentation

Project Executable Files

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 3 Marks

Step 1: Submission Checklist

S.No	Item to Submit	Submitted
1	Complete source code – app/, templates, static/, tests	Yes / To upload final package
2	README / Setup Guide	Yes
3	requirements.txt	Yes
4	.env.example	Yes; actual API key must not be submitted
5	Database / initialization logic	Yes – SQLite initialized by application
6	AI integration code	Yes – Gemini integration in recommender.py
7	Sample test inputs / screenshots	To be added
8	Deployment / run instructions	Yes

Execution Instructions

1. Open the project in VS Code.
2. Create and activate a Python virtual environment.
3. Install requirements using python -m pip install -r requirements.txt.
4. Create .env and add GOOGLE_API_KEY and GEMINI_MODEL=gemini-3.8-flash.
5. Start with python -m uvicorn app.main:app --reload.
6. Open http://127.0.0.1:8000.
7. Register, log in, open a planner, submit valid inputs and verify the result/history.

Sample Project Documentation

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: As per template

Project Title

PocketSmart AI – Your Smart Budget & Recommendation Assistant

Project Overview

PocketSmart AI is a Generative AI powered budget and recommendation assistant for Indian users. It provides structured planning for home interiors, parties and jewelry, including optional image-assisted jewelry analysis and actionable shopping links.

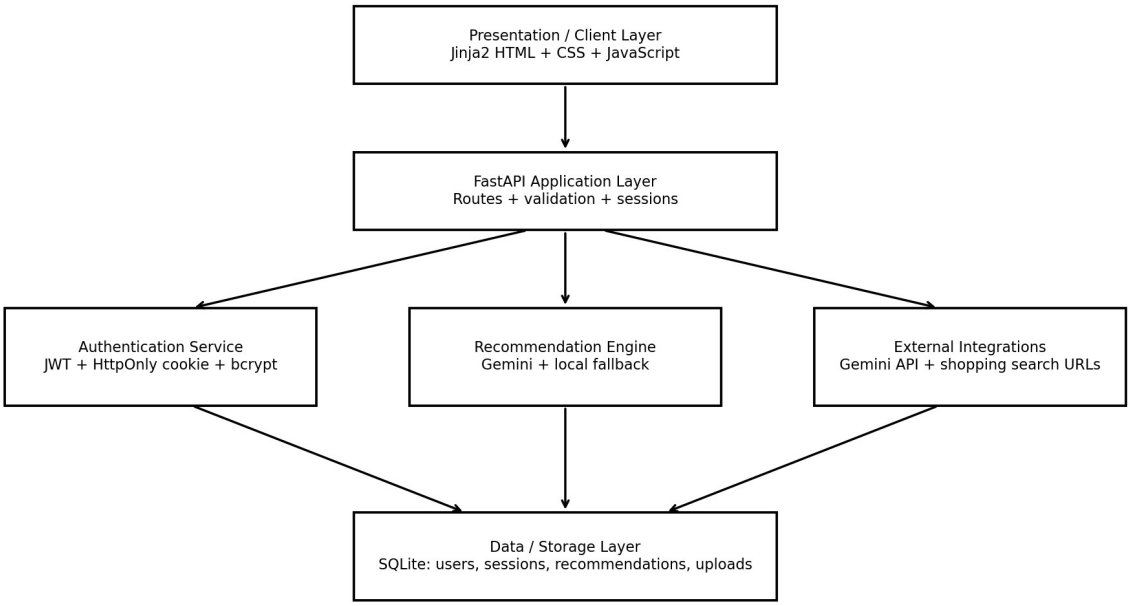
Objective

To provide budget-aware, practical and easy-to-understand recommendations using Generative AI while reducing manual browsing and planning effort.

Key Features

- Secure registration and login
- Home Interior Planner
- Party Planner
- Jewelry Planner with optional outfit image analysis
- Budget normalization and remaining-budget calculation
- Recommendation history
- Indian marketplace/service search links
- Gemini retry/model fallback and local fallback

Technical Architecture



Project Workflow

Milestone 1: Environment and API setup: Create Python environment, install dependencies, configure Gemini API key and validate application configuration.

Milestone 2: Core application: Implement FastAPI routes, authentication, database and planner schemas.

Milestone 3: AI integration: Create structured prompts, JSON extraction, budget normalization, retry and model fallback.

Milestone 4: Frontend: Build responsive templates, forms, dynamic result rendering and history UI.

Milestone 5: Testing: Validate authentication, planner APIs, budget limits, image uploads and transient Gemini failures.

Milestone 6: Demonstration and deployment: Prepare screenshots/demo, package executable source and plan future scalability.

8. Project Demonstration

Communication

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 1 Mark

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
1	Team Standup	Daily	WhatsApp / Teams / in-person	Team	Track tasks and blockers
2	Progress Update	Weekly	Online meeting / shared document	Team + mentor	Review completed work
3	Issue / Bug Discussion	As Needed	VS Code logs / chat	Developer + team	Resolve technical issues
4	Stakeholder Review	Bi-Weekly	Demo / meeting	Team + mentor	Validate features and direction
5	Final Demo Rehearsal	Once	Local environment	Team	Verify end-to-end presentation
6	Documentation Review	As Needed	Shared document	Team + mentor	Check template compliance

Communication Challenges & Resolutions

S.No	Challenge Faced	Resolution / Action Taken
1	Gemini API 503 temporary unavailability	Added retry, exponential backoff and model fallback.
2	AI output may exceed budget or contain malformed values	Added JSON extraction, validation and budget normalization.
3	Need to demonstrate image-assisted jewelry recommendation	Implemented optional image upload and multimodal Gemini input.

Demonstration of Proposed Features

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 1 Mark

S.No	Feature Name	Description	Status	Demonstrated	Remarks
1	Authentication	Register/login/ logout	Implemented	To demonstrate	Verify in final demo
2	Home Planner	Budget-based home interior plan	Implemented	To demonstrate	Observed API 200 OK during testing
3	Party Planner	Party budget and venue suggestions	Implemented	To demonstrate	Verify UI
4	Jewelry Planner	Budget/ preferences based jewelry recommendations	Implemented	To demonstrate	Verify UI
5	Image Analysis	Optional outfit image analysis	Implemented	To demonstrate	Requires Gemini availability
6	History	View previous recommendations	Implemented	To demonstrate	Verify database persistence

Metric	Value
Total Features Proposed	12 identified in development feature list
Total Features Implemented	12 identified, subject to final integrated verification
Total Features Demonstrated	To be filled after final demo
Overall Implementation Rate	To be calculated after final demo

Project Demo Planning

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 1 Mark

S.No	Demo Section	Description	Duration (mins)	Responsible Member
1	Introduction & Problem Statement	Explain user pain points and project objective	2	To be filled
2	Solution Overview	Architecture, technology stack and AI workflow	3	To be filled
3	Authentication	Register and login	2	To be filled
4	Planner Demonstration	Home, Party and Jewelry planners	7	To be filled
5	AI / Image Analysis	Show Gemini recommendation and optional outfit image	3	To be filled
6	History + Q&A	Show saved recommendations and answer questions	3	To be filled

Demo Flow Summary

Step	Activity	Notes
1	Introduction & Problem Statement	State problem, users and objective
2	Solution Overview	Show architecture and key technologies
3	Live Feature Demonstration	Run authentication and all three planners
4	Q&A Session	Explain AI, budget rules, database, security and future work

Scalability & Future Plan

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 1 Mark

S.No	Current System Limitation	Impact	Priority to Address
1	SQLite is suitable for local/small deployments but not large concurrent production workloads	Limits horizontal scaling	High
2	Gemini availability and latency are external dependencies	AI response time can vary	High
3	Shopping links are search URLs rather than live inventory/pricing integrations	Prices and stock are not synchronized	Medium

Scalability Plan

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Single/local ASGI server	Deploy behind reverse proxy with multiple workers and session-aware architecture
2	Data Storage	SQLite	Migrate to PostgreSQL and object storage for uploads
3	Performance	Synchronous external AI requests	Add async job queue/cache and request deduplication
4	Security	JWT cookie + environment API key	Add CSRF strategy, rate limiting, secret manager and production HTTPS

Future Roadmap

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	PostgreSQL, cloud object storage, production deployment	Next release	Better reliability and scalability
Phase 3	Personalized recommendation history, user preferences and spending analytics	Future release	More relevant recommendations
Phase 4	Live retailer APIs, price comparison,	Long-term	Richer shopping and accessibility experience

	multilingual voice/chat assistant		

Team Involvement in Demonstration

Date: 30 September 2026 | Team ID: To be filled | Project Name: PocketSmart AI – Your Smart Budget & Recommendation Assistant | Maximum Marks: 1 Mark

S.No	Team Member Name	Role in Demo	Section Presented	Contribution Summary	Participation
1	To be filled	Project Lead / Presenter	Introduction + Architecture	Explain objective and system flow	To be filled
2	To be filled	Developer	Authentication + Backend	Demonstrate APIs and database	To be filled
3	To be filled	AI / GenAI Presenter	Gemini + Jewelry	Explain prompts, multimodal analysis and fallback	To be filled
4	To be filled	Frontend Presenter	Planner UI	Demonstrate Home/Party/Jewelry screens	To be filled
5	To be filled	Testing / Documentation	Testing + Results	Explain test scenarios and documentation	To be filled
6	To be filled	Support / Q&A	Q&A	Answer implementation and future-scope questions	To be filled

Team Coordination Notes

Aspect	Details
Team Leader / Coordinator	To be filled
Overall Team Coordination Rating (1-5)	To be filled after demo
Any issues during demo	Potential external Gemini 503 availability; fallback logic is available
How issues were resolved	Retry/backoff, model fallback and local recommendation fallback